Oyun State Mantığı

Bu bölümde, oyun durumu (game state) kavramını derinlemesine inceleyecek, event-based ve state-based sistemler arasındaki farkları anlayacak ve bu bilgileri kullanarak oyunumuza player join/leave sistemi ile event log mekanizması ekleyeceğiz.

---

1. Game State Nedir? (Oyun Durumu)

Game state (oyun durumu), bir oyunun belirli bir andaki tüm değişkenlerinin, nesnelerinin ve oyuncularının durumlarını içeren kapsamlı veri bütünüdür. Oyunun "anlık fotoğrafı" olarak düşünülebilir.

1.1. Game State'in Bileşenleri

Bileşen	Açıklama	Örnek
Oyuncu Durumları	Her oyuncunun konumu, canı, envanteri, yetenekleri	`player.x = 125.5`, `player.health = 80`
Dünya Nesneleri	Haritadaki statik ve dinamik nesneler	Kapıların açık/kapalı olması, sandıkların içeriği
Oyun Metrikleri	Skor, süre, tur sayısı, oyun modu	`score = 5-3`, `timeLeft = 180`
Sistem Durumları	Bağlantı durumları, sunucu zamanı, tick sayısı	`connectedPlayers = 8`, `serverTick = 12345`
Geçici Efektler	Patlamalar, ışınlanmalar, animasyon durumları	`explosionAt = (x, y, z)`, `particleSystem.active = True`

1.2. State'in Özellikleri

1. **Deterministik (Belirleyici):** Aynı başlangıç durumu ve aynı input'lar ile her zaman aynı sonuca ulaşılır.
2. **Kapsamlı (Comprehensive):** Oyunun o anki tüm önemli bilgilerini içerir.
3. **Değişken (Mutable):** Her tick'te input'lara göre güncellenir.
4. **Senkronize Edilebilir (Synchronizable):** Ağ üzerinden istemcilere iletilebilir.

1.3. Oyunlarda State Yönetimi Örnekleri

Oyun Türü	State Yönetimi
FPS (Counter-Strike)	Her oyuncunun konumu, silahı, canı, mermi sayısı, bomba durumu, tur skoru
MOBA (League of Legends)	Her oyuncunun seviyesi, eşyaları, yetenek soğuma süreleri, can ve mana değerleri, kule ve minyon durumları
MMORPG (World of Warcraft)	Oyuncu konumu, can, mana, buff/debuff'lar, envanter, görev ilerlemesi, dünya boss'larının durumu

Oyun Türü	State Yönetimi
Strateji (Age of Empires)	Her birimin konumu, canı, saldırı gücü, binaların durumu, kaynak miktarları, teknoloji seviyeleri

2. Event-based vs State-based Sistemler

Oyun mantığını ve ağ senkronizasyonunu tasarlarken iki temel yaklaşım vardır: Event-based (olay tabanlı) ve State-based (durum tabanlı).

2.1. Event-based Sistem (Olay Tabanlı)

Event-based sistemde, oyun durumundaki değişiklikler "olay" (event) olarak tanımlanır ve bu olaylar ağ üzerinden iletilir. İstemciler, bu olayları alarak kendi yerel durumlarını günceller.

Nasıl Çalışır?

```
Oyuncu Ateş Etti → "SHOOT" event'i oluşturulur → Sunucuya gönderilir
Sunucu event'i alır → İşler (örneğin hasar hesaplar) → "DAMAGE_DEALT"
event'i oluşturur
Sunucu, "DAMAGE_DEALT" event'ini tüm istemcilere gönderir
İstemciler event'i alır → Kendi durumlarını günceller (can azalır, efekt
oynatılır)
```

Avantajları	Dezavantajları
Bant genişliği verimlidir: Sadece değişiklikler gönderilir.	**Durum tutarlılığı zordur:** Bir event kaybolursa, durum tutarsız olur.
Hızlıdır: Sadece olan değişiklikler işlenir.	**Sıralama kritiktir:** Event'ler doğru sırada işlenmelidir.
Gerçekçi his verir: Olaylar anlık olarak algılanır.	**Kod karmaşıktır:** Event'leri yönetmek, sıralamak ve doğrulamak zordur.
Hata ayıklama kolaydır: Event log'ları ile sorunlar izlenebilir.	**Geç bağlanan oyuncular:** Geçmiş event'leri bilmez, durumu yakalaması zordur.

Event-based Örnekler:

- Mermi atışları
- Hasar verme
- Yetenek kullanımı
- Kapı açma/kapama
- Sohbet mesajları

2.2. State-based Sistem (Durum Tabanlı)

State-based sistemde, oyunun tüm durumu (state) düzenli aralıklarla (tick) istemcilere gönderilir. İstemciler, aldıkları durumu kendi yerel durumlarının üzerine yazar.

Nasıl Çalışır?

```
Sunucu (Her Tick'te):
    1. Tüm oyuncuların input'larını işle
    2. Oyun dünyasını güncelle (fizik, çarpışma)
    3. Güncel FULL_STATE'i oluştur
    4. Tüm istemcilere gönder

İstemci (Her Tick'te):
    1. Sunucudan gelen FULL_STATE'i al
    2. Yerel durumu tamamen güncelle
    3. Render et
```

Avantajları	Dezavantajları
Durum tutarlılığı garantidir: Her tick'te tüm durum gelir.	**Bant genişliği yüksektir:** Değişmese bile tüm durum gönderilir.
Basit kod: İstemci sadece gelen veriyi uygular.	**Geçikmeli his verebilir:** Tick rate ile sınırlıdır.
Hata toleransı yüksektir: Bir paket kaybolsa bile sonraki tick düzeltir.	**Ölçeklenebilir değildir:** Oyuncu sayısı arttıkça paket boyutu büyür.
Geç bağlanan oyuncular: Hemen güncel durumu alabilir.	**Anlık olaylar geç algılanır:** Bir atış olayı, state ile birlikte gelir.

State-based Örnekler:

- Oyuncu konumları
- Can ve enerji değerleri
- Skor tablosu
- Nesne durumları

2.3. Hibrit Yaklaşım (Event + State)

Modern oyunların çoğu, her iki yaklaşımı birleştirir:

- **State-based:** Oyuncu konumları, can değerleri, skor gibi sürekli değişen veriler için.
- **Event-based:** Atış, yetenek kullanımı, kapı açma gibi anlık eylemler için.

```
Örnek Akış:
1. Oyuncu Ateş Eder → "SHOOT" event'i sunucuya gönderilir (Event-based)
2. Sunucu hasarı hesaplar → Oyuncu B'nin canını günceller (State değişir)
3. Sonraki tick'te sunucu, FULL_STATE ile tüm oyuncuların güncel can
değerlerini gönderir (State-based)
4. İstemciler, FULL_STATE'i alır ve can değerlerini günceller
```

---

3. Uygulama: Player Join/Leave Sistemi Ekleme

Şimdi, bir önceki dersteki Full State sunucumuzu geliştirerek, oyuncuların katılım ve ayrılışlarını daha iyi yöneten bir sistem ekleyelim.

3.1. Sunucu: Gelişmiş Player Join/Leave Yönetimi

```
import socket
import json
import time
from datetime import datetime

HOST = '127.0.0.1'
PORT = 12345
BUFFER_SIZE = 4096

class AdvancedGameServer:
    def __init__(self):
        self.server_socket = socket.socket(socket.AF_INET,
socket.SOCK_DGRAM)
        self.server_socket.bind((HOST, PORT))
        self.clients = {}  # {addr: {"id": int, "x": float, "y": float,
"name": str, "joinTime": float, "lastUpdate": float}}
        self.next_player_id = 1
        self.tick_rate = 30
        self.tick_interval = 1.0 / self.tick_rate
        self.tick_counter = 0
        self.event_log = []  # Tüm olayları tutar
        self.max_log_entries = 1000
        print(f"Gelişmiş Sunucu {HOST}:{PORT} adresinde çalışıyor...")
        print(f"Tick Rate: {self.tick_rate} Hz")

    def log_event(self, event_type, player_id, details, addr=None):
        """Olayları log dosyasına ve belleğe ekler."""
        event = {
            "timestamp": datetime.now().isoformat(),
            "tick": self.tick_counter,
            "type": event_type,
            "playerId": player_id,
            "address": str(addr) if addr else None,
            "details": details
        }
        self.event_log.append(event)

        # Max log boyutunu koru
        if len(self.event_log) > self.max_log_entries:
            self.event_log.pop(0)

        # Konsola yazdır
        print(f"[{event['timestamp'][11:19]}] {event_type.upper()}: Player
{player_id} - {details}")

        return event

    def run(self):
```

```
        """Ana sunucu döngüsü."""
        last_tick_time = time.time()

        while True:
            current_time = time.time()

            # Tick işlemleri
            if current_time - last_tick_time >= self.tick_interval:
                self.tick()
                last_tick_time = current_time
                self.tick_counter += 1

            # Non-blocking mesaj alma
            self.server_socket.settimeout(0.005)
            try:
                data, addr = self.server_socket.recvfrom(BUFFER_SIZE)
                self.handle_message(data, addr, current_time)
            except socket.timeout:
                continue
            except Exception as e:
                print(f"Hata: {e}")

    def handle_message(self, data, addr, current_time):
        """Gelen mesajı işler."""
        try:
            message = json.loads(data.decode('utf-8'))
            msg_type = message.get('type')

            if msg_type == 'JOIN':
                self.handle_join(message, addr, current_time)

            elif msg_type == 'POSITION_UPDATE':
                self.handle_position_update(message, addr, current_time)

            elif msg_type == 'DISCONNECT':
                self.handle_disconnect(message, addr)

            elif msg_type == 'GET_STATE':
                self.broadcast_full_state()

            elif msg_type == 'GET_LOG':
                # İstemci event log istiyor
                self.handle_log_request(message, addr)

        except json.JSONDecodeError:
            print(f"Geçersiz JSON: {data}")
        except Exception as e:
            print(f"Mesaj işleme hatası: {e}")

    def handle_join(self, message, addr, current_time):
        """Yeni oyuncu katılımını işler."""
        # Eğer bu adres zaten kayıtlıysa, önce çıkar
        if addr in self.clients:
            old_id = self.clients[addr]["id"]
```

```
            self.log_event("REJOIN", old_id, f"Oyuncu yeniden katıldı (eski
ID: {old_id})", addr)
            del self.clients[addr]

        # Yeni oyuncu oluştur
        player_id = self.next_player_id
        self.next_player_id += 1

        player_name = message.get('name', f"Player_{player_id}")

        self.clients[addr] = {
            "id": player_id,
            "name": player_name,
            "x": 0.0,
            "y": 0.0,
            "joinTime": current_time,
            "lastUpdate": current_time
        }

        # Katılım olayını log'la
        self.log_event("JOIN", player_id, f"'{player_name}' katıldı", addr)

        # Yeni oyuncuya ID ve mevcut durumu gönder
        response = {
            "type": "ASSIGN_ID",
            "playerId": player_id,
            "tickRate": self.tick_rate,
            "playerName": player_name
        }
        self.server_socket.sendto(json.dumps(response).encode('utf-8'),
addr)

        # Tüm oyunculara full state gönder (yeni oyuncuyu dahil et)
        self.broadcast_full_state()

    def handle_position_update(self, message, addr, current_time):
        """Konum güncellemesini işler."""
        if addr in self.clients:
            self.clients[addr]["x"] = message.get('x', 0.0)
            self.clients[addr]["y"] = message.get('y', 0.0)
            self.clients[addr]["lastUpdate"] = current_time

            # Hareket log'u (sadece önemli mesafelerde logla)
            # Oyuncu 5 birimden fazla hareket ettiyse log'la
            # Bu örnekte basitlik için sadece her 30 tick'te bir
log'layalım
            if self.tick_counter % 30 == 0:
                self.log_event("MOVE", self.clients[addr]["id"],
                               f"X={self.clients[addr]['x']:.1f}, Y=
{self.clients[addr]['y']:.1f}", addr)

    def handle_disconnect(self, message, addr):
        """Oyuncu ayrılmasını işler."""
        if addr in self.clients:
```

```
            player_id = self.clients[addr]["id"]
            player_name = self.clients[addr]["name"]

            # Ayrılma olayını log'la
            self.log_event("LEAVE", player_id, f"'{player_name}' ayrıldı",
addr)

            # Oyuncuyu listeden çıkar
            del self.clients[addr]

            # Kalan oyunculara durumu gönder
            self.broadcast_full_state()

    def handle_log_request(self, message, addr):
        """İstemciye event log'u gönderir."""
        count = message.get('count', 50)  # Son kaç olay
        filtered_log = self.event_log[-count:]

        response = {
            "type": "LOG_RESPONSE",
            "count": len(filtered_log),
            "events": filtered_log
        }

        try:
            self.server_socket.sendto(json.dumps(response).encode('utf-8'),
addr)
            print(f"Log gönderildi: {len(filtered_log)} olay")
        except Exception as e:
            print(f"Log gönderme hatası: {e}")

    def broadcast_full_state(self):
        """Tüm oyun durumunu FULL olarak tüm istemcilere gönderir."""
        if len(self.clients) == 0:
            return

        # Full state paketini oluştur
        players_list = []
        for addr, player_data in self.clients.items():
            players_list.append({
                "id": player_data["id"],
                "name": player_data["name"],
                "x": player_data["x"],
                "y": player_data["y"],
                "isActive": True
            })

        full_state = {
            "type": "FULL_STATE_UPDATE",
            "tick": self.tick_counter,
            "timestamp": int(time.time() * 1000),
            "playerCount": len(players_list),
            "players": players_list,
            "gameState": {
```

```
                "status": "playing",
                "totalTicks": self.tick_counter,
                "maxPlayers": 50
            }
        }

        state_bytes = json.dumps(full_state).encode('utf-8')
        state_size = len(state_bytes)

        # Tüm istemcilere gönder
        for addr in self.clients.keys():
            try:
                self.server_socket.sendto(state_bytes, addr)
            except Exception as e:
                print(f"Broadcast hatası ({addr}): {e}")

        # Her 30 tick'te bir debug bilgisi
        if self.tick_counter % 30 == 0:
            print(f"[Tick {self.tick_counter}] Broadcast:
{len(self.clients)} oyuncu, {state_size} byte")

    def tick(self):
        """Her tick'te yapılacak işlemler."""
        # Zaman aşımı kontrolü (30 saniye güncellenmemiş oyuncuları
temizle)
        current_time = time.time()
        timeout_seconds = 30.0
        disconnected = []

        for addr, player_data in self.clients.items():
            if current_time - player_data["lastUpdate"] > timeout_seconds:
                disconnected.append(addr)
                self.log_event("TIMEOUT", player_data["id"],
                               f"'{player_data['name']}' zaman aşımına
uğradı", addr)

        for addr in disconnected:
            del self.clients[addr]

        # Eğer oyuncu ayrıldıysa state gönder
        if disconnected:
            self.broadcast_full_state()
        elif len(self.clients) > 0:
            # Normal tick'te state gönder
            self.broadcast_full_state()

if __name__ == "__main__":
    server = AdvancedGameServer()
    try:
        server.run()
    except KeyboardInterrupt:
        print("\nSunucu kapatılıyor...")
        server.server_socket.close()
```

4. Uygulama: Event Log Sistemi Ekleme (İstemci)

İstemci, sunucudan gelen event log'larını görüntüleyebilecek.

```
import socket
import json
import threading
import time
import keyboard

HOST = '127.0.0.1'
PORT = 12345
BUFFER_SIZE = 4096

class AdvancedGameClient:
    def __init__(self):
        self.client_socket = socket.socket(socket.AF_INET,
socket.SOCK_DGRAM)
        self.player_id = None
        self.player_name = ""
        self.local_x = 0.0
        self.local_y = 0.0
        self.players = {}
        self.running = True
        self.tick_rate = 30
        self.tick_interval = 1.0 / self.tick_rate
        self.last_state_tick = 0
        self.state_received = False
        self.log_events = []
        self.show_log = False
        self.log_page = 0

        # Kullanıcı adı al
        self.player_name = input("Kullanıcı adınız: ").strip() or
f"Player_{self.player_id}"

        # Sunucuya bağlan
        self.connect_to_server()

    def connect_to_server(self):
        """Sunucuya JOIN mesajı gönderir ve ID bekler."""
        join_message = {
            "type": "JOIN",
            "name": self.player_name
        }
        self.client_socket.sendto(json.dumps(join_message).encode('utf-8'),
(HOST, PORT))
        print(f"Sunucuya bağlanılıyor... ({self.player_name})")

        # ID almak için bekle
        self.client_socket.settimeout(3.0)
```

```
        try:
            data, _ = self.client_socket.recvfrom(BUFFER_SIZE)
            response = json.loads(data.decode('utf-8'))
            if response.get('type') == 'ASSIGN_ID':
                self.player_id = response['playerId']
                self.tick_rate = response.get('tickRate', 30)
                self.tick_interval = 1.0 / self.tick_rate
                print(f"Oyuncu ID: {self.player_id} atandı. (Tick Rate:
{self.tick_rate})")
            else:
                print("Beklenmeyen yanıt:", response)
                self.running = False
        except socket.timeout:
            print("Sunucudan yanıt alınamadı!")
            self.running = False
        except Exception as e:
            print(f"Bağlantı hatası: {e}")
            self.running = False

    def send_position(self):
        """Mevcut konumu sunucuya gönderir."""
        if self.player_id is None:
            return

        position_message = {
            "type": "POSITION_UPDATE",
            "x": self.local_x,
            "y": self.local_y
        }

        try:
            self.client_socket.sendto(
                json.dumps(position_message).encode('utf-8'),
                (HOST, PORT)
            )
        except Exception as e:
            print(f"Konum gönderme hatası: {e}")

    def request_log(self, count=20):
        """Sunucudan event log'u ister."""
        log_request = {
            "type": "GET_LOG",
            "count": count
        }
        try:
            self.client_socket.sendto(
                json.dumps(log_request).encode('utf-8'),
                (HOST, PORT)
            )
            print("Log isteği gönderildi...")
        except Exception as e:
            print(f"Log isteği hatası: {e}")

    def receive_messages(self):
```

```
        """Sunucudan gelen mesajları sürekli dinler."""
        self.client_socket.settimeout(0.05)
        while self.running:
            try:
                data, _ = self.client_socket.recvfrom(BUFFER_SIZE)
                message = json.loads(data.decode('utf-8'))
                self.handle_server_message(message)
            except socket.timeout:
                continue
            except json.JSONDecodeError:
                continue
            except Exception as e:
                print(f"Alım hatası: {e}")
                break

    def handle_server_message(self, message):
        """Sunucudan gelen mesajı işler."""
        msg_type = message.get('type')

        if msg_type == 'FULL_STATE_UPDATE':
            self.handle_full_state(message)

        elif msg_type == 'ASSIGN_ID':
            self.player_id = message['playerId']
            self.player_name = message.get('playerName', self.player_name)
            print(f"Oyuncu ID güncellendi: {self.player_id}")

        elif msg_type == 'LOG_RESPONSE':
            self.handle_log_response(message)

    def handle_full_state(self, message):
        """Full state mesajını işler."""
        self.last_state_tick = message.get('tick', 0)
        players = message.get('players', [])

        # Tüm oyuncuları güncelle
        self.players.clear()
        for player in players:
            self.players[player['id']] = {
                'name': player.get('name', f"Player_{player['id']}"),
                'x': player['x'],
                'y': player['y']
            }

        # Eğer kendi ID'miz listede yoksa (anomaly), kendimizi ekleyelim
        if self.player_id not in self.players:
            self.players[self.player_id] = {
                'name': self.player_name,
                'x': self.local_x,
                'y': self.local_y
            }

        self.state_received = True
```

```
        # Log modunda değilsek debug yazdır
        if not self.show_log:
            if self.last_state_tick % 5 == 0:
                self.debug_print_state()

    def handle_log_response(self, message):
        """Log yanıtını işler."""
        self.log_events = message.get('events', [])
        print(f"\n--- Event Log ({len(self.log_events)} olay) ---")
        for event in self.log_events:
            timestamp = event.get('timestamp', '')[11:19]
            event_type = event.get('type', 'UNKNOWN').upper()
            player_id = event.get('playerId', '?')
            details = event.get('details', '')
            print(f"[{timestamp}] {event_type:10} | Player {player_id:4} |
{details}")
        print("--- Log Sonu ---")

    def debug_print_state(self):
        """Mevcut oyun durumunu konsola yazdırır."""
        print("\033c", end="")
        print(f"=== OYUN DURUMU ===")
        print(f"Oyuncu: {self.player_name} (ID: {self.player_id})")
        print(f"Tick: {self.last_state_tick} | Ben: X={self.local_x:.1f},
Y={self.local_y:.1f}")
        print(f"Aktif Oyuncular: {len(self.players)}")
        print("\n--- Oyuncu Listesi ---")
        for pid, data in sorted(self.players.items()):
            is_me = "⭐" if pid == self.player_id else "  "
            print(f"{is_me} Oyuncu {pid:3} ({data['name']:12}): X=
{data['x']:6.1f}, Y={data['y']:6.1f}")
        print("\n[WASD] Hareket | [Q] Çıkış | [L] Log Göster | [R] Log
İste")

    def show_log_screen(self):
        """Log ekranını gösterir."""
        print("\033c", end="")
        print(f"=== EVENT LOG (Sunucu) ===")
        print(f"Oyuncu: {self.player_name} (ID: {self.player_id})")
        print("L tuşuna basarak ana ekrana dönebilirsiniz.")
        print("R tuşu ile log yenilenir.\n")

        if not self.log_events:
            print("Henüz log alınmadı. 'R' tuşuna basarak log isteyin.")
        else:
            for i, event in enumerate(self.log_events[-30:], 1):  # Son 30
olay
                timestamp = event.get('timestamp', '')[11:19]
                event_type = event.get('type', 'UNKNOWN').upper()
                player_id = event.get('playerId', '?')
                details = event.get('details', '')
                print(f"{i:3}. [{timestamp}] {event_type:10} |
P{player_id:3} | {details}")
```

```
        print("\n[R] Yenile | [L] Ana Ekrana Dön")

    def handle_input(self):
        """Klavye girdilerini işler."""
        move_speed = 3.0
        last_send_time = 0
        log_requested = False

        while self.running:
            current_time = time.time()

            # Log modu aktifse farklı input işleme
            if self.show_log:
                if keyboard.is_pressed('l'):
                    self.show_log = False
                    time.sleep(0.2)
                    continue
                if keyboard.is_pressed('r'):
                    self.request_log(30)
                    time.sleep(0.3)
                self.show_log_screen()
                time.sleep(0.1)
                continue

            # Normal oyun modu
            moved = False

            # Hareket inputları
            if keyboard.is_pressed('w'):
                self.local_y += move_speed * self.tick_interval
                moved = True
            if keyboard.is_pressed('s'):
                self.local_y -= move_speed * self.tick_interval
                moved = True
            if keyboard.is_pressed('a'):
                self.local_x -= move_speed * self.tick_interval
                moved = True
            if keyboard.is_pressed('d'):
                self.local_x += move_speed * self.tick_interval
                moved = True

            # L tuşu ile log moduna geç
            if keyboard.is_pressed('l'):
                self.show_log = True
                self.request_log(30)
                time.sleep(0.3)
                continue

            # R tuşu ile log iste
            if keyboard.is_pressed('r') and not log_requested:
                self.request_log(20)
                log_requested = True
                time.sleep(0.3)
            if not keyboard.is_pressed('r'):
```

```
                log_requested = False

            # Q ile çıkış
            if keyboard.is_pressed('q'):
                print("Çıkılıyor...")
                self.disconnect()
                break

            # Konumu tick rate'de gönder
            if moved or (current_time - last_send_time >=
self.tick_interval):
                self.send_position()
                last_send_time = current_time

            time.sleep(0.01)

    def disconnect(self):
        """Sunucudan ayrılma mesajı gönderir ve bağlantıyı kapatır."""
        self.running = False
        if self.player_id is not None:
            disconnect_message = {"type": "DISCONNECT"}
            try:
                self.client_socket.sendto(
                    json.dumps(disconnect_message).encode('utf-8'),
                    (HOST, PORT)
                )
                print("Ayrılma mesajı gönderildi.")
            except:
                pass
        self.client_socket.close()
        print("Bağlantı kapatıldı.")

    def run(self):
        """İstemciyi çalıştırır."""
        if not self.running:
            return

        print("\nİstemci başlatıldı!")
        print("L tuşu ile event log ekranını açabilirsiniz.")
        print("Q tuşuna basarak çıkabilirsiniz.")
        time.sleep(1)

        # Mesaj alma thread'i
        receive_thread = threading.Thread(target=self.receive_messages)
        receive_thread.daemon = True
        receive_thread.start()

        # Input işleme (ana thread)
        try:
            self.handle_input()
        except KeyboardInterrupt:
            print("\nKullanıcı tarafından durduruldu.")
        finally:
            self.disconnect()
```

```
if __name__ == "__main__":
    client = AdvancedGameClient()
    client.run()
```

---

5. Sistemin Özellikleri

5.1. Join/Leave Sistemi

- **JOIN:** Yeni oyuncu katıldığında log oluşturur ve tüm oyunculara duyurur.
- **LEAVE:** Oyuncu ayrıldığında log oluşturur ve durumu günceller.
- **REJOIN:** Aynı adres tekrar bağlanırsa (bağlantı kopup geldiyse) log oluşturur.
- **TIMEOUT:** Oyuncu 30 saniye güncellenmezse otomatik olarak ayrı sayılır.

5.2. Event Log Sistemi

- Tüm önemli olaylar (JOIN, LEAVE, MOVE, TIMEOUT) log'a eklenir.
- Log'lar zaman damgası ve tick numarası ile saklanır.
- İstemci, log'ları görüntüleyebilir ve yenileyebilir.
- Log boyutu sınırlandırılmıştır (max 1000 olay).

5.3. Event-based ve State-based Karışımı

Event-based (Anlık)	State-based (Sürekli)
JOIN, LEAVE (Katılım/Ayrılma olayları)	FULL_STATE (Tüm oyuncu konumları)
MOVE log'u (Hareket olayları)	Oyuncu listesi (Her tick güncellenir)
TIMEOUT (Zaman aşımı olayları)	Oyun metrikleri (Skor, süre)
LOG_REQUEST/RESPONSE (Log talep/yanıt)	ASSIGN_ID (ID ataması)

---

6. İleri Çalışma

1. **Log Filtreleme:** İstemciye log filtresi ekleyin. Sadece JOIN/LEAVE veya sadece MOVE olaylarını gösterebilsin.
2. **Log Export:** Sunucu, log'ları bir dosyaya (`.csv` veya `.json`) yazsın. Sunucu kapatıldığında log'lar kaybolmasın.
3. **Player List GUI:** Konsol yerine, oyuncuları bir tablo formatında gösterin (sütunlar: ID, İsim, X, Y, Durum).
4. **Event-based Hareket:** Konum güncellemelerini sadece hareket olduğunda (event olarak) gönderin. State-based ile karşılaştırın.
5. **Olay Sıralaması:** Her olaya bir sequence number ekleyin. İstemci, olayları bu sıraya göre işlesin.

Özet

- **Game State:** Oyunun anlık durumu. Tüm oyuncuları, nesneleri ve metrikleri içerir.
- **Event-based Sistem:** Değişiklikler olay olarak iletilir. Bant genişliği verimli, durum tutarlılığı zor.

- **State-based Sistem:** Tüm durum düzenli aralıklarla gönderilir. Tutarlılık kolay, bant genişliği yüksek.
- **Hibrit Yaklaşım:** Modern oyunlar her ikisini birleştirir. Anlık olaylar event, sürekli veriler state ile.
- **Join/Leave Sistemi:** Oyuncu katılım ve ayrılma olaylarını yönetir. Log'lama ve durum güncellemesi gerektirir.
- **Event Log:** Oyun içi olayların kaydıdır. Hata ayıklama, analiz ve denetim için kullanılır.

Bu sistem, gerçek bir oyunun state yönetiminin temelini oluşturur. Event log'lar, hem geliştiriciler için hata ayıklama hem de oyuncular için şeffaflık sağlar. Join/Leave sistemi ise çok oyunculu bir oyunun olmazsa olmazıdır.